BTC Price Predictor

[Names of all group members], 26 October 2026

Live website: [link to the live website] · **Code:** github.com/tjommilaskus/ml-innlevering_2

1: DESCRIBE THE PROBLEM

SCOPE

The Bitcoin (BTC) price moves a lot: on average 0.4% every hour, and often 10% or more in a week. People who own a little BTC want to know *where the price is going* and *how much it could move*. Today they get answers from price targets on social media, "technical analysis" indicators or prediction markets, and these answers rarely come with an honest record of how often they were right.

Our goal is a website, the **BTC Price Predictor**, that answers both questions for the next hour, the next 7 days and the next 30 days, and is honest about how good the answers are. For each horizon it shows a guessed (average) price and direction, the expected size of the move, an 80% range the price should land in, and whether the guess is one of the model's *confident* ones. The second half of the site is a test bench: the user picks a period in the past, and the site replays the model using only the prices it could have known at the time, with a chart, every guess and a "report card" with the hit rate and the chance that the result is just luck. The site is meant to be fun, and to teach a lesson about markets: the size of a move can be predicted quite well, the direction barely.

Why machine learning? Without machine learning, the usual answers are rules of thumb: "the price stays where it is" (the best guess if prices are a random walk), "it went up, so it keeps going up" (momentum), or a range based on how much the price moved lately. These rules are our baselines. Prices are close to unpredictable (Fama, 1970), but the *size* of moves comes in clusters: calm hours follow calm hours, wild days follow wild days (Cont, 2001). A model trained on eight years of hourly prices can combine many such signals (recent moves, volume, time of day) and weigh them better than a fixed rule, and the machine learning workflow gives us a clean way to test whether it does.

Users and business value. The users are curious BTC owners and students, not professional traders. The site is free, so its value lies in visits and trust: people come back if the ranges hold what they promise. A trading product would need a direction signal strong enough to pay for trading fees; we test this in Section 3 and do not find one.

The system. The model is one part of a small pipeline: a data module downloads hourly prices from the Binance API into a CSV file, a feature module turns them into one row per hour, two regression models are trained and calibrated, and a Flask web app serves the results. Changes ripple through: if Binance is down, the forecasts get older (the site then uses its saved prices), and a changed feature means the models, the range calibration and the website text must all be checked again.

Resources. No special hardware is needed: a model trains in under a second on a laptop CPU, and the site runs on a free cloud host. The work was done by [number] students over about [number] weeks.

METRICS

- **Direction:** the share of guesses with the right direction (up or down), compared with a coin flip (50%) and with "always up". A few hundred guesses can beat 50% by luck, so we also report a one-sided binomial test: the chance of getting at least this many right by flipping a coin ("chance it's luck"). Under 5% means the result is probably real.
- **Price accuracy:** 100% minus the mean absolute percentage error, compared with guessing the last price. Users see this first, but it is misleading on its own: any guess close to the last price scores over 99% for the next hour.
- **Move size and range:** how much better the expected move is than always guessing the average move (1 − MAE of the model / MAE of the average; above 0% is better), and how often the real price lands inside the 80% range (it should be close to 80%).
- **Software:** the time it takes to load the page and to run the test bench.

The range and move metrics are tied to the business goal: users trust the site if the ranges hold what they promise. **We count the project as a success if** we reach (1) range coverage between 75% and 85% on the test period for all horizons, (2) a move guess better than the average move, (3) an hourly direction hit rate above 50% with a chance of luck under 5%, and (4) a front page that loads in about a second and a full test in under 10 seconds. Making money after fees is a stretch goal.

2: DATA

Source. We use hourly BTC/USDT candles (open, high, low and close price, and traded volume) from Binance, the largest crypto exchange, through its free public market data API (`data-api.binance.vision`, endpoint `/api/v3/klines`), which needs no account (Binance, 2026). We use every hour from 1 January 2018: 77,158 hours until 26 October 2026. The prices are saved in `btc_1h.csv`, and the website fetches new hours as they finish.

Labels. The label comes from the data itself, so no manual labelling is needed: for a guess made at hour *t* with a horizon of *H* hours, the label is the average closing price over the next *H* hours divided by the last price, minus one. For the hourly horizon this is the next hour's change. We use the *average* price for weeks and months because it is what a user can act on (spreading a purchase over the period) and it is less noisy than a single hour. The labels are exact exchange prices, and they are consistent: one exchange, one trading pair, one clock (UTC).

Cleaning. 122 hours are missing, in 27 gaps (the longest is 33 hours). We insert empty rows for them, so a "last 24 hours" window never jumps over a gap; period averages accept up to half the hours missing; rows with a missing feature are dropped. The hour still in progress is never used.

Features. Each hour gets one row with two groups of features:

- *Change model* (direction): the % change of the average price and the log change of the average volume, for each of the last 4 periods.
- *Move model* (size of the move): the log of the average absolute hourly change in each of the last 4 periods, and over the last 24 hours, 7 days and 30 days; the log of the average high–low range; and, for hours only, the hour of day and day of week (one-hot).

Volatility measures are heavily skewed, so we use their logarithms. All features are standardized inside the model pipeline, fitted on the training data only.

Privacy and ethics. The data is public market data with no personal information, and the site stores nothing about visitors except a visitor counter. The main ethical risk is that someone treats a guess as financial advice. The page says clearly that it is not, always shows the uncertainty next to the guess, and shows how often the model was wrong in the past.

Is there enough data? For hours, yes: the test period alone has about 50,000 guesses. For weeks and months, no: the test period has about 300 weeks and 70 months, and Bitcoin's history cannot be made longer. This turns out to be the main limit of the project.

3: MODELING

HOW WE TEST

If a price model has seen any price from the period it guesses, its results look far better than they will be on the website. We therefore test **walk-forward**, the time-series version of cross-validation (Hyndman & Athanasopoulos, 2021): the model re-learns at the start of every month, only from periods that were completely over by then, and guesses every period in that month. The website works the same way.

Prices from 2018 until mid-2019 are only used for learning (Figure 1). We compared models and chose settings on the **validation period** (July 2019 – December 2020), and ran the **test period** (January 2021 – 26 October 2026) once with the chosen settings. Hourly guesses are made every hour, weekly guesses every Monday and monthly guesses on the 1st (covering 30 days).



Figure 1: The hourly BTC price, with the validation and test periods.

THE MODEL

Each horizon has two regressions with separate features. The **change model** guesses the % change of the average price; its sign is the direction. The **move model** guesses the logarithm of the size of the move (move sizes are very skewed), scaled so that its average matches the average move in the training data. The 80% range is the 10th to 90th percentile of the change model's training errors, measured in units of the expected move, so it is wide in wild markets and narrow in calm ones. A guess is **confident** when the guessed change is large compared with the expected move, with the cut-off set so that 20% of the training guesses are confident.

We use **ridge regression** (Hoerl & Kennard, 1970) from scikit-learn (Pedregosa et al., 2011): it is fast, stable when features are correlated (as the changes of neighbouring periods are), and its coefficients can be read directly.

BASELINES AND MODELS WE TRIED

We compared the model with simple baselines and with three other models in exactly the same setup: plain linear regression, ridge with a much stronger penalty (alpha = 1000 instead of 1), and gradient boosting (HistGradientBoostingRegressor, based on LightGBM; Ke et al., 2017), which can learn non-linear patterns. The validation period has only 19 months, too few to compare anything, so Table 1 shows hours and weeks.

Table 1: Validation period. Direction = right direction; range = real price inside the 80% range; move skill = improvement over always guessing the average move.

Method	Hour: direction	Hour: range	Hour: move skill	Week: direction	Week: range	Week: move skill
Always up	51.1%	–	–	59.5%	–	–
Same direction as last period	45.0%	–	–	49.4%	–	–
Opposite of last period	**55.0%**	–	–	50.6%	–	–
Last period's move	–	–	−6.2%	–	–	−36.2%
Linear regression	54.1%	80.3%	**+18.8%**	55.7%	76.0%	**+8.0%**

Method	Hour: direction	Hour: range	Hour: move skill	Week: direction	Week: range	Week: move skill
Ridge, alpha = 1 (chosen)	54.1%	80.3%	**+18.8%**	55.7%	76.0%	**+8.0%**
Ridge, alpha = 1000	54.3%	80.2%	+18.8%	53.2%	77.2%	+5.9%
Gradient boosting	52.2%	78.1%	+18.2%	59.5%	30.4%	−5.6%

Ridge and linear regression give identical results: with tens of thousands of training rows and at most 39 features, the penalty hardly matters, and even alpha = 1000 changes little. Gradient boosting is worse on almost everything, and its ranges are far too narrow for weeks (30% coverage; 5% for months): it fits the training data so closely that its training errors are much smaller than its errors on new data, so ranges built from them are useless. For the hourly direction, the rule "guess the opposite of the last hour" was as good as any model. We chose ridge with alpha = 1, as good as the best model on every measure that worked, and simpler.

We also tried looking back 1, 2, 4, 8 and 16 periods. Four was best for weeks (55.7% direction, +8.0% move skill); for hours, one period was slightly better on direction (54.9% against 54.1%), with the same move skill. We kept four for all horizons, so the website uses one setting. The full table is in `experiments.ipynb`.

RESULTS ON THE TEST PERIOD

Table 2: Final results on the test period (1 January 2021 – 26 October 2026).

	Hour	Week	Month
Guesses	50,927	303	69
Right direction (chance it's luck)	**52.6%** (< 0.01%)	47.5% (82%)	58.0% (11%)
Baseline: always up	50.5%	50.8%	58.0%
Baseline: opposite of last period	52.7%	51.5%	49.3%
Right direction, confident guesses	55.0% (of 11,207)	49.0% (of 96)	60.0% (of 15)
Real price inside the 80% range	**79.4%**	**75.9%**	**79.7%**
Move skill vs. the average move	**+22.6%**	**+10.2%**	−6.9%
Baseline: last period's move	−2.1%	−21.7%	−26.7%
Price accuracy	99.62%	96.33%	91.80%
Baseline: same as last price	99.62%	96.44%	92.49%

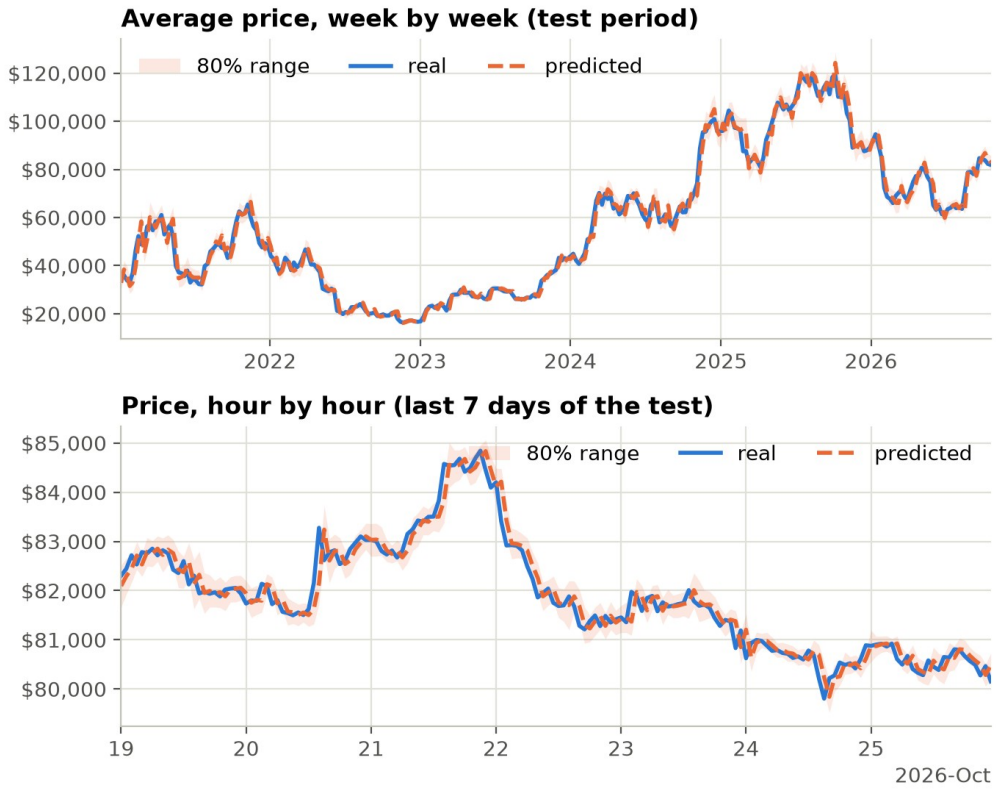


Figure 2: Guessed and real prices on the test period. The guess stays close to the last price, so the dashed line looks like the real line moved one step to the right.

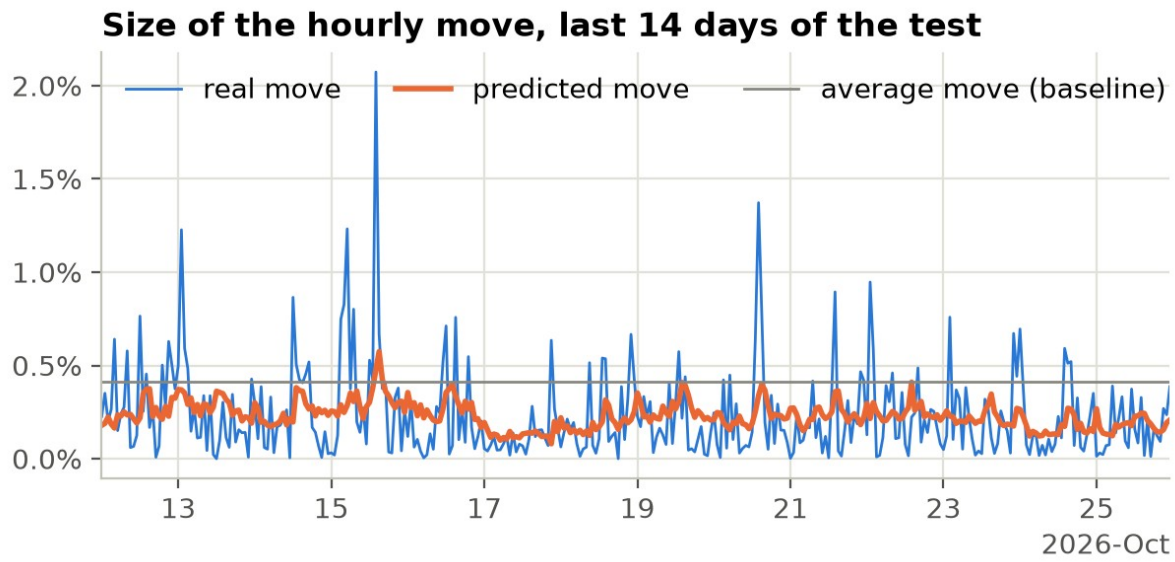


Figure 3: The expected size of the hourly move follows the calm and wild stretches of the market; the baseline is a flat line.

Against our success criteria:

- **(1) Range coverage between 75% and 85%: reached** for all horizons (79.4%, 75.9%, 79.7%), with weeks close to the limit.
- **(2) Move guess better than the average: reached for hours (+22.6%) and weeks (+10.2%), not for months (−6.9%).** The "last period's move" baseline is worse than the average for all three.

- **(3) Hourly direction better than a coin flip: reached**, 52.6% over 50,927 guesses and 55.0% for confident guesses, with a chance of luck far below 5%. But "opposite of the last hour" scores 52.7%, so the model is no better than this simple rule.
- **(4) Speed: reached** (Section 4).

For weeks and months, the direction is no better than luck, and the price accuracy is slightly *worse* than guessing the last price. With so few guesses, small changes move the results a lot: in an earlier version of our code, where weekly guesses were spaced 7 days apart from 1 January 2021 instead of starting on Mondays, the hit rates were 51.5% for weeks and 47.1% for months.

WHAT THE MODEL LOOKS AT

The features are standardized, so the size of a ridge coefficient shows how much the model relies on a feature (Figure 4). For hours, the change model puts negative weights on the price changes of the last two hours: after a rise, it guesses a small fall. This *mean reversion* is the whole hourly direction edge, and it is exactly what the "opposite of last period" baseline does. For weeks and months, the weights flip sign from one period to the next, a sign of fitting noise.

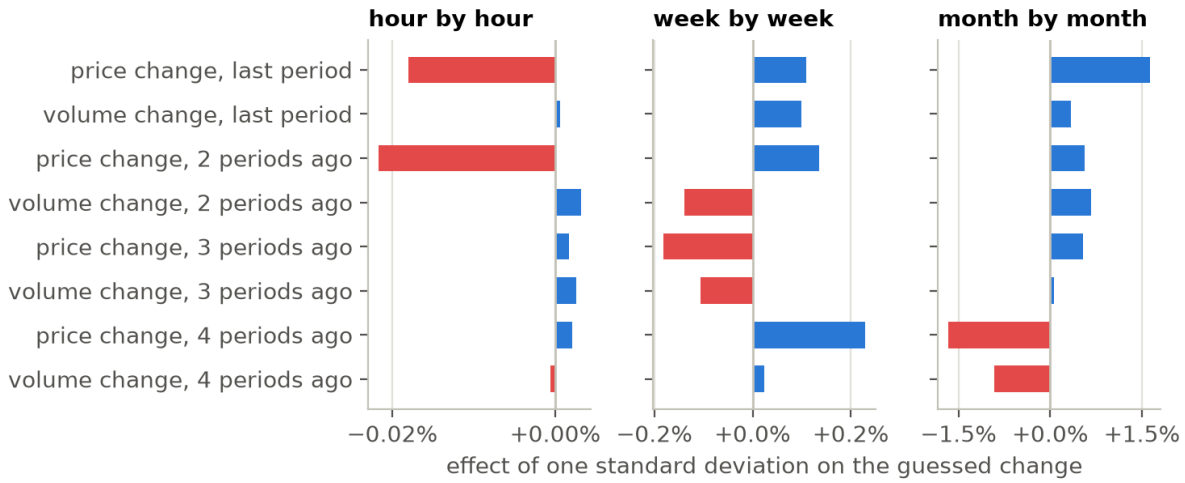


Figure 4: Change model coefficients (trained on all data until 26 October 2026). Blue pushes the guess up, red down.

The move model relies most on the high–low range of the last period and the average move over the last 24 hours and 7 days: big moves follow big moves, the volatility clustering known from finance (Cont, 2001). The time of day matters too (Figure 5): the hourly move is 0.83 times the usual size at 03:00 UTC and 1.29 times at 14:00 UTC, around the time US markets open and US economic news comes out.

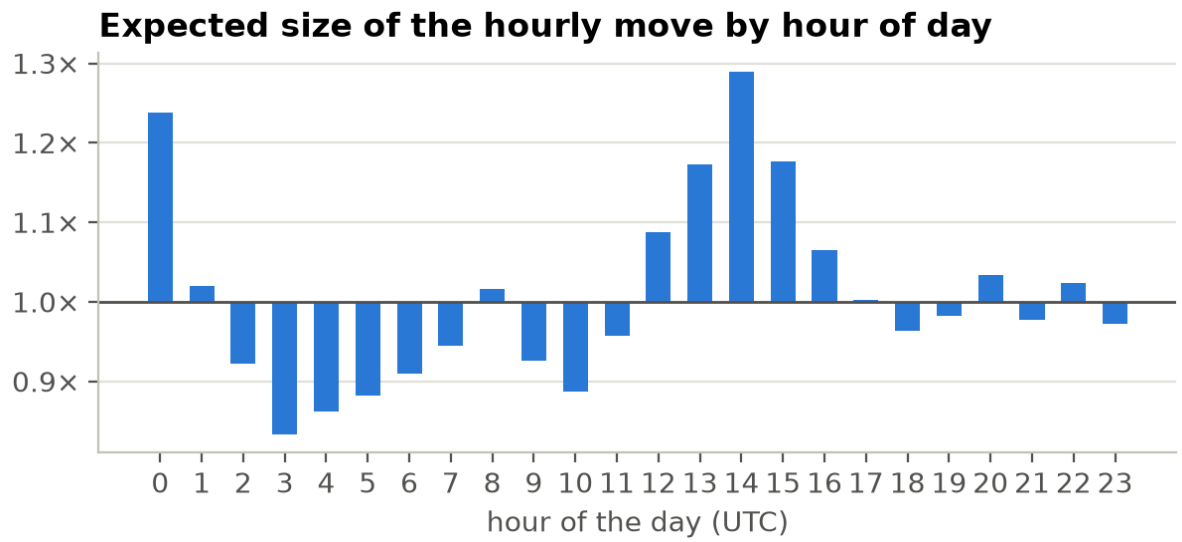


Figure 5: Expected size of the hourly move by hour of day, from the move model.

WHERE IT GOES WRONG

One lucky year? No: the hourly hit rate is between 51.6% and 53.5% in every year of the test, and 53–58% for confident guesses (Figure 6). The range coverage is between 78.7% and 80.7% every year.

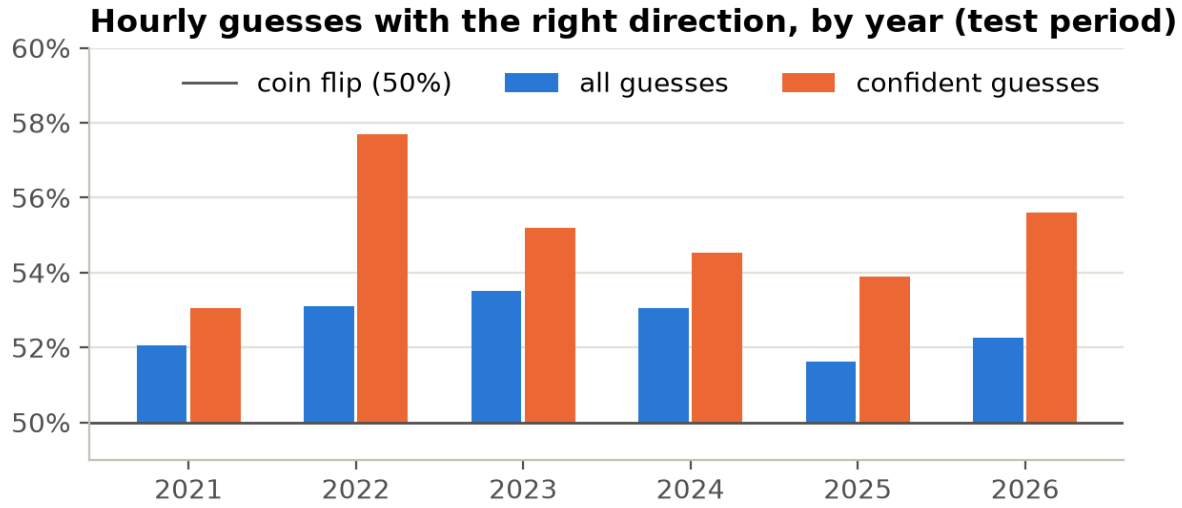


Figure 6: Hourly guesses with the right direction, by year. The bars start at the coin-flip line.

Calm and wild markets. Split into three equal groups by the expected move, the hourly range holds the real price 80.0%, 79.2% and 79.0% of the time in calm, normal and wild markets, so the ranges adapt as intended. For weeks it is 73.3%, 77.2% and 77.2%: a little too narrow in calm weeks.

The biggest misses. The eight weeks with the biggest misses were all sudden shocks, with the real price far outside the 80% range. Six of them followed well-known news: the collapse of the crypto lender Celsius (June 2022, −20%), the FTX collapse (November 2022, −14%), Tesla dropping Bitcoin payments and China's crackdown (May 2021, −10% and −14%), Tesla's Bitcoin purchase (February 2021, +19%) and the rally after the US bank failures (March 2023, +16%). A model that only sees prices cannot see such news coming.

Could you make money? We tested a simple strategy: buy when the model says up, sell short when it says down, and exit at the end of the period, paying about 0.2% per round trip (Binance, 2026). The hourly model earns about 0.002% per trade before fees, even though it is right 52.6% of the time: it is right more often on small moves, so its wins are smaller than its losses. After fees, it loses about 0.2% per trade (Table 3). The monthly "profit" comes from guessing up in 80% of months while BTC mostly went up, the same as "always up". The model gives no edge for trading; its value is in the move size and the ranges.

Table 3: Simple trading test on the test period, with a 0.2% round-trip fee.

Guesses	Trades	Guessed up	Gain per trade	After fees
Hour, all	50,927	61%	+0.002%	−0.198%
Hour, confident only	11,207	73%	+0.001%	−0.199%
Week, all	303	88%	−0.127%	−0.327%
Month, all	69	80%	+1.736%	+1.536%

4: DEPLOYMENT

How it is deployed. The website is a Flask app (`app.py`) that uses the same modules as the notebooks (`btc_data.py` and `btc_model.py`), so the site shows exactly what this report measures. It runs under the gunicorn web server in a Docker container, and the repository contains a `render.yaml` blueprint for Render's free plan. We use Render's Frankfurt region because Binance blocks some countries, such as the US. The README describes how to run the site locally and how to deploy it.

What happens on a visit. The site fetches new hours from Binance at most every 10 minutes. When a new hour has finished, it trains the three models on all prices so far and keeps the forecasts until the next hour. The test bench runs the walk-forward test for the period the user picks. If Binance cannot be reached, the site keeps working with its saved prices, and bad input gives an error message instead of a crash.

Performance. Measured in the production container on a laptop, the first visit after a new hour takes 1.8 seconds (training the three models), and later visits are answered at once. The test bench takes 0.4 seconds for the default view (the last 7 days, hour by hour), about 3 seconds for every week or month since 2021, and 8.3 seconds for the heaviest request, every hour since 2021. The table shows at most the latest 500 guesses, so no page is larger than about 130 kB. The app uses at most about 360 MB of memory, within the free plan's 512 MB. The free plan only has a fraction of a CPU, so long tests are slower on the live site, and the site sleeps after 15 minutes without visitors, so the first visit after a quiet period takes up to a minute.

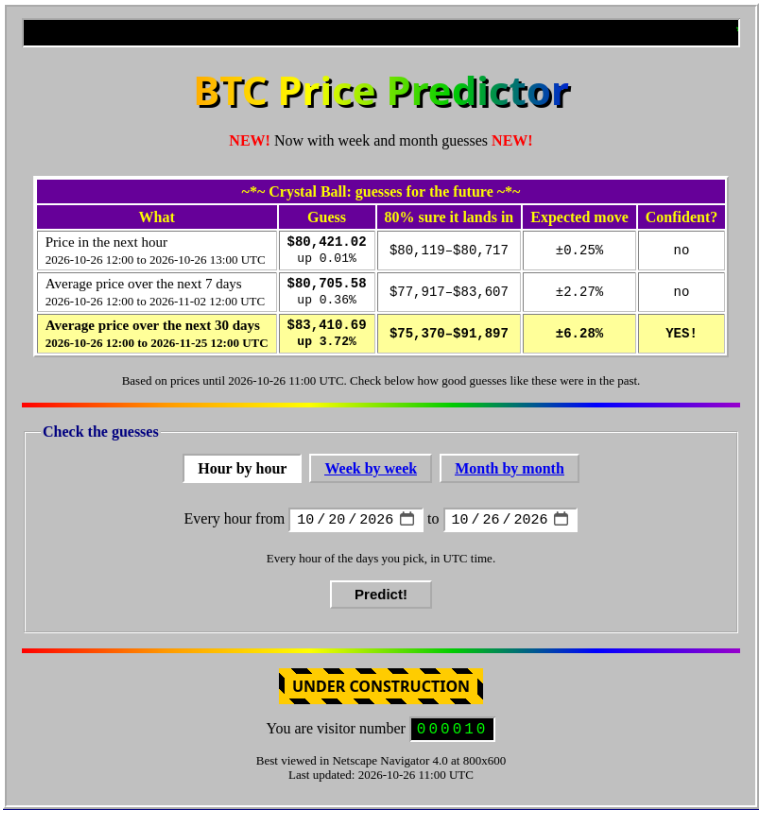


Figure 7: The front page, with the forecasts and the form for testing the model on past data.

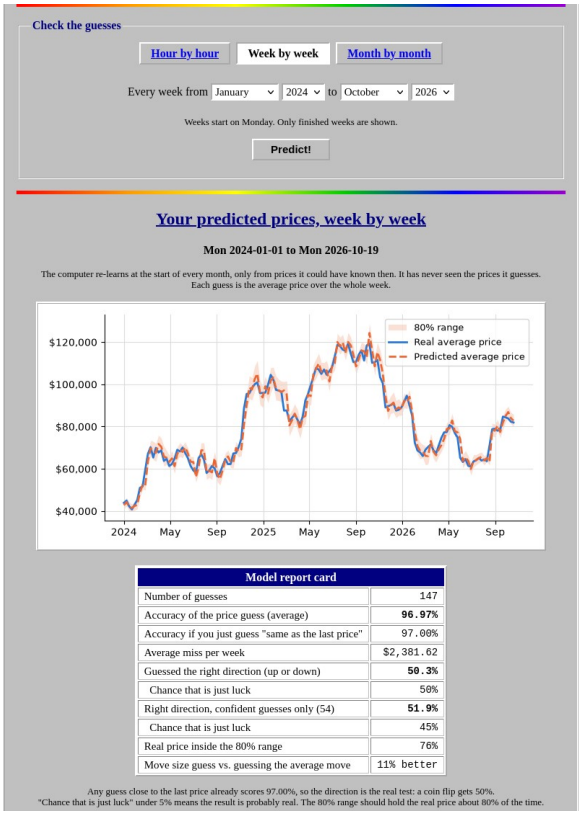


Figure 8: The test bench, week by week from January 2024, with the chart and the report card. A table with every guess follows below.

Monitoring and maintenance. The model retrains itself, so normal changes in the market are handled automatically. For a real product we would add:

- **Scoring of live forecasts:** save every forecast, score it when the period is over, and raise an alert if the range coverage leaves 75–85% or the move skill drops below zero.
- **Data alerts**, for example when Binance has not answered for an hour.
- **A yearly drift check** with the by-year table (Figure 6).

Improvements. The most promising next steps are data the price chart does not have (options-implied volatility and futures funding rates for the move model, and news for the large shocks), quantile regression or GARCH (Bollerslev, 1986) for wider tails in the ranges, and more coins, which would also give the week and month models more data.

5: REFERENCES

- Binance (2026). *Spot API documentation: market data endpoints.* https://developers.binance.com/docs/binance-spot-api-docs/rest-api/market-data-endpoints
- Binance (2026). *Fee schedule.* https://www.binance.com/en/fee/schedule
- Bollerslev, T. (1986). Generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics*, 31(3), 307–327.
- Cont, R. (2001). Empirical properties of asset returns: stylized facts and statistical issues. *Quantitative Finance*, 1(2), 223–236.
- Fama, E. F. (1970). Efficient capital markets: a review of theory and empirical work. *The Journal of Finance*, 25(2), 383–417.
- Harris, C. R. et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362.
- Hoerl, A. E. & Kennard, R. W. (1970). Ridge regression: biased estimation for nonorthogonal problems. *Technometrics*, 12(1), 55–67.
- Hunter, J. D. (2007). Matplotlib: a 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. The website's chart code follows the Matplotlib example "Embedding in a web application server": https://matplotlib.org/stable/gallery/user_interfaces/web_application_server_sgskip.html
- Hyndman, R. J. & Athanasopoulos, G. (2021). *Forecasting: principles and practice* (3rd ed.), section 5.10. OTexts. https://otexts.com/fpp3/tscv.html
- Ke, G. et al. (2017). LightGBM: a highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, 30.
- McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 56–61.
- Pedregosa, F. et al. (2011). Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Virtanen, P. et al. (2020). SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17, 261–272.
- Documentation for Flask (https://flask.palletsprojects.com), gunicorn (https://gunicorn.org), Docker (https://docs.docker.com) and Render (https://render.com/docs).
- **AI tools:** Claude Code (Anthropic) was used as a coding assistant for parts of the code (among them the experiments notebook, the deployment files and changes to the website) and to draft this report. All numbers come from the code in the repository. [GROUP: describe exactly how you used AI tools.]